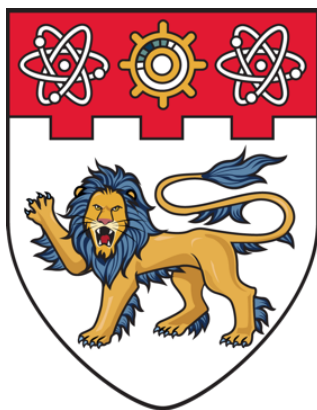




**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

AI6126 ADVANCED COMPUTER VISION

**PROJECT REPORT ON:
OUTPUT SPACE UDA AND LMM TRANSFER**

**SUBMITTED BY:
XIE CHEN (G2502685C)
YU FANYUE (G2503500B)
ZHANG YU (G2503065J)**

1 Introduction

In this project, we implemented two distinct transfer learning paradigms: Unsupervised Domain Adaptation (UDA) based on self-training strategies and Knowledge Transfer in Large Multimodal Models (LMMs) using adapter modules.

In Task I, we evaluated Vanilla Self-Training (ST), Class-Balanced Self-Training (CBST) [5], and Confidence Regularized Self-Training (CRST)[6] to explore the characteristics and limitations of employing pseudo-labeling mechanisms for UDA.

In Task II, we shifted our focus to the CLIP pre-trained model. By comparing the feature reshaping mechanism of CLIP-Adapter[1] with the non-parametric cache mechanism of Tip-Adapter[4], we analyzed the performance and underlying principles of knowledge transfer in both domain generalization and few-shot learning scenarios.

2 Task I: Benchmarking Self-Training Strategies in UDA

2.1 Compared Methods

2.1.1 Vanilla Self-Training

In UDA tasks, self-training establishes a supervision mechanism via pseudo-labeling. Unlike adversarial learning, which requires a discriminator, self-training can be formulated as an Expectation-Maximization (EM)[3] optimization process. During the iterative cycles, the model performs the E-step by estimating the most likely classes for target domain samples and then the M-step by updating parameters based on self-training loss. This process is grounded in the Entropy Minimization principle[2], which encourages the model to produce deterministic predictions on the target domain.

To ensure the generated categorical pseudo-labels are as accurate as possible, it is essential to employ an 'easy-to-hard' scheme via self-paced curriculum learning[5]. For the downstream image classification task, the objective function can be formulated as a combination of supervised source loss and self-supervised target loss, subject to a confidence-based selection mask (Equation 1):

$$\mathcal{L}_{total} = \mathbb{E}_{(x,y) \sim \mathcal{D}_s} [-\log p(y|x)] + \mathbb{E}_{x \sim \mathcal{D}_t} [\mathbb{I}(\max p \geq \lambda) \cdot (-\log p(\hat{y}|x))] \quad (1)$$

where $\mathcal{D}_s$ and $\mathcal{D}_t$ denote the source and target domains, $p(y|x)$ is the predicted class probability, $\mathbb{I}(\cdot)$ is the indicator function and $\hat{y}$ represents the generated hard pseudo-label for the target sample.

Compared to the original framework proposed in CBST[5], we simplify the pixel-wise summation to an image-level objective while maintaining the hard-labeling constraint $\hat{\mathbf{y}}_t \in \{\mathbf{e}^{(i)}\} \cup \mathbf{0}$.

2.1.2 Class-balanced Self-training

Since Vanilla Self-Training employs a uniform threshold λ to control pseudo-label selection, it poses a risk of overlooking the different adaptation difficulties across different categories, potentially leading to insufficient learning for hard examples. To address this issue, Zou et al.[5] proposed Class-Balanced Self-Training (CBST), which establishes class-specific confidence thresholds for pseudo-label selection instead of a uniform one. The loss function can be formulated as Equation 2:

$$\mathcal{L}_{CBST} = \mathbb{E}_{(x,y) \sim \mathcal{D}_s} [-\log p(y|x)] + \mathbb{E}_{x \sim \mathcal{D}_t} \left[\sum_{c=1}^C \mathbb{I}(c = \arg \max p, p \geq e^{-\lambda_c}) \cdot (-\log p(c|x) + \lambda_c) \right] \quad (2)$$

where $\mathcal{D}_s, \mathcal{D}_t$ denote the source and target domains, C is the total number of classes, λ_c represents the class-specific threshold, $\mathbb{I}(\cdot)$ is the indicator function that enforces the self-paced learning strategy.

2.1.3 Confidence Regularized Self-training

Despite the adoption of self-paced learning, the accuracy of pseudo-labels in Self-Training still cannot be ensured. Treating these pseudo-labels directly as hard labels can easily lead to overconfident mistakes. To address this issue, Zou et al. proposed Confidence Regularized Self-Training (CRST)[6] to prevent infinite entropy minimization. This framework incorporates both label regularization (to generate soft pseudo-labels) and model regularization (to encourage output smoothness).

The loss function of CRST can be expressed as the CBST loss plus a regularization term (Equation 3 to 5):

$$\mathcal{L}_{CRST} = \underbrace{\mathcal{L}_S + \mathcal{L}_T}_{\text{CBST Loss}} + \underbrace{\mathcal{R}_{MR}}_{\text{Regularization}} \quad (3)$$

$$\mathcal{L}_S = \mathbb{E}_{(x,y) \sim \mathcal{D}_s} [-\log p(y|x)] \quad (4)$$

$$\mathcal{L}_T = \mathbb{E}_{x \sim \mathcal{D}_t} \left[\sum_{c=1}^C \hat{y}_c^{(\alpha)} \cdot (-\log p(c|x) + \lambda_c) \right] \quad (5)$$

In the LRENT (Label Regularization)[6] variant, the target domain loss utilizes soft pseudo-labels $\hat{y}_c^{(\alpha)}$ instead of hard labels, and the soft pseudo-labels are generated by regularized confidence based on the parameter α (Equation 6).

$$\hat{y}_c^{(\alpha)} = \frac{(p(c|x)/\lambda_c)^{1/\alpha}}{\sum_{k=1}^C (p(k|x)/\lambda_k)^{1/\alpha}} \quad (6)$$

Besides, three distinct regularization strategies are introduced in Model Regularization (MR) [6] terms. The strategies can be formulated as Equation 7 to Equation 9:

$$\mathcal{R}_{MRKLD} = -\frac{1}{C} \sum_{c=1}^C \log p(c|x) \quad (7)$$

$$\mathcal{R}_{MRENT} = \sum_{c=1}^C p(c|x) \log p(c|x) \quad (8)$$

$$\mathcal{R}_{MRL2} = \sum_{c=1}^C p(c|x)^2 \quad (9)$$

2.2 Experimental Setup

We conducted UDA experiments using the aforementioned self-training models across three benchmarks: Amazon $\rightarrow$ Webcam (Office-31), Art $\rightarrow$ Real-World (Office-Home), and Photo $\rightarrow$ Sketch (PACS).

For the downstream classification task, we utilized the ResNet-50 architecture from torchvision as our backbone. The model was first fine-tuned on the source domain before starting the self-training process.

We employed Cosine Annealing learning rate scheduler with an initial value of 1e-5 in training process. The self-training procedure was conducted for a maximum of 50 rounds, with each round consisting of 2 training epochs. To implement the curriculum learning strategy, we set the initial selection ratio to 0.1, increasing it by 0.02 per round, up to a maximum threshold of 0.8. Furthermore, an early stopping mechanism was employed to capture the optimal performance for each method. For the LRENT configuration, we set the label smoothing factor α to 0.05, and model regularization terms were integrated into the total loss after being scaled by a coefficient of 0.001.

We conducted an additional set of comparative experiments on the Photo $\rightarrow$ Sketch (PACS) dataset. Two distinct initialization strategies for the ResNet backbone were evaluated: one where the model was fully

fine-tuned on the source domain, and another where early stopping was applied once the source validation accuracy reached 70%. These pre-trained models then served as the starting points for subsequent self-training.

2.3 Experimental Results

The final experimental results are summarized in Table 1. The results demonstrate that all self-training methods achieve positive gains, with the sole exception of the Vanilla ST result on the first Photo $\rightarrow$ Sketch set, where a slight performance degradation is observed compared to the source-only baseline. CBST consistently outperforms Vanilla ST on the Amazon $\rightarrow$ Webcam and Art $\rightarrow$ Real-World tasks, although it yields inferior results on the second Photo $\rightarrow$ Sketch set. Furthermore, while certain CRST variants surpass CBST on both the Art $\rightarrow$ Real-World and Photo $\rightarrow$ Sketch datasets, all CRST configurations fall short of the performance achieved by CBST on the Amazon $\rightarrow$ Webcam task.

Synthesizing the above results, it is evident that no single method achieves optimal performance across all experimental scenarios, however, both CBST and CRST demonstrate an overall superiority over the vanilla ST method. Furthermore, on datasets with significant domain gaps such as Photo $\rightarrow$ Sketch, although applying early stopping during source-domain fine-tuning can enhance the final UDA performance, the overall results achieved by self-training methods remain unsatisfactory.

Table 1: Performance Comparison of Self-Training Methods (Top-1 Acc)

Method	Amazon $\rightarrow$ Webcam	Art $\rightarrow$ Real World	Photo $\rightarrow$ Sketch	Photo $\rightarrow$ Sketch (ES)
Source Acc	90.96%	79.42%	99.40%	70.36%
Source Only	73.21%	69.31%	16.11%	27.59%
Vanilla ST	77.36%	72.11%	15.96%	31.84%
CBST	90.19%	73.22%	20.74%	30.75%
LRENT	88.93%	72.99%	21.33%	31.92%
MRENT	88.55%	73.58%	20.67%	32.45%
MRKLD	89.31%	74.34%	19.01%	32.45%
MRL2	86.16%	72.76%	20.31%	31.94%
LRENT+MRKLD	84.78%	73.35%	21.10%	32.12%

2.4 Performance Analysis

The experiment results reveal the inherent complexity of applying ST methods across varying domain gaps. The experiments on Photo $\rightarrow$ Sketch suggest that excellent performance on the source domain may lead to overfitting, which makes it harder for the model to generalize to the target domain. The model may become overly reliant on features unique to the source domain, leading to a failure in capturing effective pseudo-labels during the ST process. In such scenarios, moderately reducing source-domain performance via early stopping can yield a better initialization for the target domain.

Furthermore, the comparative results between CBST and CRST emphasize that the choice of regularization should be tailored to specific domain characteristics. In tasks like Amazon $\rightarrow$ Webcam, where the domain gap lies in backgrounds, the model’s confidence is often reliable; thus, excessive regularization may degrade discriminative power. Conversely, in more challenging tasks, CRST provides more stable gains by mitigating the overconfident errors that arise from larger domain shifts.

Moreover, compared to adversarial UDA methods based on CycleGAN, self-training approaches involve iterative training only on the downstream task model, resulting in much lower computational resource requirements. However, we also observed that in scenarios with substantial domain gaps, UDA methods operating solely in the output space remain quite limited in effectiveness. Such challenges cannot be easily resolved through class-balancing or regularization techniques alone. When the model fails to generate reliable pseudo-labels, hybrid UDA approaches that integrate Input Space and Feature Space adaptations may provide a more promising path forward.

3 Task II: Benchmarking Knowledge Transfer in LMMs

3.1 Compared Methods

3.1.1 CLIP-Adapter

In contrast to traditional transfer learning paradigms, CLIP-based transfer learning shifts away from predicting fixed categorical labels. Instead, it learns semantic alignment by maximizing the similarity between matched image-text pairs[1]. The pre-trained CLIP model is frozen to serve as a robust feature extractor, and a CLIP-Adapter[1] module can be utilized to facilitate downstream task specialization.

The CLIP-Adapter module employed in this experiment utilizes a residual bottleneck architecture consisting of two linear layers and an activation function to generate the adapted features. The adapted features are fused with the original features via a residual connection, where a residual ratio α controls the degree to which the original knowledge is retained. The integrated feature f_{new} can be formulated as:

$$f_{new} = \alpha \cdot \phi(f_i W_1) W_2 + (1 - \alpha) \cdot f_i \quad (10)$$

where W_1 and W_2 represent the weight matrices of the linear layers, and ϕ denotes the activation function.

The loss function employed during the training of the adapter module is formulated as Equation 11:

$$L_{total} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log \left(\frac{e^{z_{i,c}}}{\sum_{j=1}^C e^{z_{i,j}}} \right) \quad (11)$$

where N denotes the number of samples in the current training batch, C represents the total number of target classes in the dataset, $y_{i,c}$ indicates the one-hot encoded ground truth label, $z_{i,c}$ refers to the Logits produced by the model.

3.1.2 Tip-Adapter

Tip-Adapter[4] introduces a non-parametric cache model to store features from downstream tasks, mitigating the distribution gap between pre-trained models and target domains. This mechanism gives Tip-Adapter the capability for effective prediction without requiring extensive fine-tuning on the target domain[4].

The cache model uses the frozen CLIP image encoder to form the cache keys (F_{train}). The corresponding ground-truth labels are utilized as the cache values (L_{train}). During inference, Tip-Adapter retrieves knowledge by calculating the affinity between the test feature and the stored cache keys. The resulting prediction logits z can be defined by the following equation:

$$z = \alpha \cdot \exp(-\beta(1 - f_{test} F_{train}^T)) L_{train} + f_{test} W_t^T \quad (12)$$

where α represents the blending scale, β signifies the sharpness modulation factor, f_{test} is the d -dimensional feature vector of the test image, F_{train}^T denotes the transposed matrix of cached training features, L_{train} represents the matrix of ground-truth labels for the cached samples, and W_t^T signifies the transposed matrix of textual embeddings generated by the CLIP text encoder.

While the aforementioned process is training-free, the Tip-Adapter-F variant can further enhance performance by treating the initialized F_{train} as a set of learnable parameters. The cache keys can be guided to evolve from original sample features into more discriminative features. The loss function of the process can be expressed as Equation 13:

$$L_{total} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log \left(\frac{e^{z_{i,c}}}{\sum_{j=1}^C e^{z_{i,j}}} \right) \quad (13)$$

3.2 Experimental Setup

In this experiment, three datasets were employed to evaluate the effectiveness of the adapter methods: Office-31, Office-Home and PACS.

For the downstream classification task, we utilized the ResNet-50 architecture from CLIP as our backbone. In the CLIP-Adapter experiments, we evaluated the performance under both zero-shot and 3-shot settings directly on the target domain. Furthermore, we assessed the effectiveness of initializing the adapter with weights fine-tuned on the source domain (as mentioned in the Self-Training section) before testing on the target domain. For the Tip-Adapter experiments, we inherited the 3-shot configuration from the CLIP-Adapter setup to evaluate both the training-free and few-shot methods.

In terms of parameter configurations, the residual ratio for CLIP-Adapter was set to 0.01, while for Tip-Adapter, α and β were set to 1.0 and 5.5, respectively. For all fine-tuning processes, we employed a cosine annealing scheduler with an initial learning rate of 1×10^{-3} . The maximum number of training epochs was set to 20, supplemented by an early stopping strategy.

3.3 Experimental Results

The final experimental results are summarized in the Table 2. It can be observed that for CLIP-Adapter, fine-tuning on a dataset with semantic similarities to the target domain can enhance zero-shot performance. However, if few-shot data is available in the target domain, performing few-shot learning directly on that domain yields the optimal results, whereas fine-tuning on other datasets may instead lead to a performance degradation. Regarding Tip-Adapter, its training-free method consistently outperforms the original CLIP zero-shot results, and its fine-tuning mode achieves superior performance compared to the few-shot results of CLIP-Adapter across all settings.

Table 2: Comparison of Transfer Performance (Top-1 Acc)

Method	Office-31(Webcam)	Office-Home(Real World)	PACS(Sketch)
CLIP-Adapter (T-0-shot)	67.92%	82.86%	78.90%
CLIP-Adapter (T-3-shot)	87.55%	85.31%	85.85%
CLIP-Adapter (S-fine-tuned)	77.61%	84.19%	79.94%
CLIP-Adapter (S-fine-tuned + T-3-shot)	83.40%	85.54%	83.94%
Tip-Adapter (Training-free)	72.83%	83.57%	79.54%
Tip-Adapter (Fine-tuned)	89.81%	86.07%	86.74%

3.4 Performance Analysis

Regarding the analysis of CLIP-Adapter’s bottleneck structure, it functions by learning to map CLIP’s general features into a more task-specific classification space. When fine-tuning on a source domain semantically similar to the target domain, the discriminative information from the similar domain can, to some extent, rectify the classification bias of the original CLIP model. However, for a lightweight architecture like an Adapter, the specific noise from similar domains inevitably exerts an influence; thus, the precise information provided by a few samples from the target domain is clearly more effective than a large volume of similar but distinct data.

Besides, we attribute the robust and superior performance of Tip-Adapter to its mechanism of conducting a ‘nearest neighbor search’ via $f_{test}F_{train}^T$, rather than forcibly reshaping the feature distribution as CLIP-Adapter does. By preserving the original CLIP predictions and superimposing a correction term based on sample similarity, this ‘residual compensation’ approach is significantly less prone to negative transfer than ‘feature reshaping’.

Compared to traditional Unsupervised Domain Adaptation methods, CLIP/Tip-Adapter approaches offer a significant advantage in terms of faster convergence and reduced computational resource requirements

during fine-tuning. Furthermore, knowledge transfer approaches based on LMM semantics can achieve results that better align with expectations on datasets like Art $\rightarrow$ Real-World, where the primary domain gap lies in object shapes. However, we also noticed that such transfer learning methods are heavily dependent on the quality of the pre-trained model. If the target domain contains information that falls outside the pre-trained distribution of CLIP, the performance tends to be poor no matter how adapter architecture is employed.

4 Conclusion

Based on the experimental results and comparative analysis of self-training strategies and multimodal model transfer techniques, we can draw the following conclusions:

1. CBST and CRST generally outperform traditional Vanilla ST. In simpler tasks where the domain gap primarily involves background color variations, CBST demonstrates particularly superior performance. However, in complex scenarios with significant domain shifts (e.g., Photo $\rightarrow$ Sketch), CRST effectively mitigates the overfitting caused by noisy pseudo-labels through the introduction of confidence regularization. Additionally, moderate early stopping on the source domain provides a more generalizable weight initialization, though we found that even with this optimization, the performance of pure ST methods remains unsatisfactory when faced with extreme domain gaps.
2. Compared to adversarial UDA methods based on CycleGAN, self-training approaches offer an advantage in terms of computational efficiency. However, UDA methods operating solely in the Output Space remain limited. Our analysis suggests that when a model fails to generate reliable pseudo-labels, hybrid UDA approaches may be more promising than simple class-balancing or regularization.
3. Tip-Adapter exhibits significant superiority in CLIP fine-tuning. Compared to CLIP-Adapter, which forcibly reshapes the feature space mapping, Tip-Adapter utilizes a "residual compensation" mechanism via cached sample retrieval. This approach achieves more robust performance gains, especially in few-shot scenarios.
4. Knowledge transfer based on LMM semantics achieves results that better align with expectations on datasets like Art $\rightarrow$ Real-World, where the primary gap lies in object shapes, outperforming traditional UDA methods in capturing semantic consistency. However, their performance ceiling remains heavily dependent on the original distribution of the pre-trained model.

5 Acknowledgments

The report was polished with the assistance of Google Gemini 3 for language refinement.

The implementation in this project was based on the following codebases:

ST: <https://github.com/yzou2/CRST>

CLIP-Adapter: <https://github.com/gaopengcuhk/CLIP-Adapter>

Tip-Adapter: <https://github.com/gaopengcuhk/Tip-Adapter>

References

- [1] GAO, P., GENG, S., ZHANG, R., MA, T., FANG, R., ZHANG, Y., LI, H., AND QIAO, Y. Clip-adapter: Better vision-language models with feature adapters. *International journal of computer vision* 132, 2 (2024), 581–595.
- [2] GRANDVALET, Y., AND BENGIO, Y. Semi-supervised learning by entropy minimization. *Advances in neural information processing systems* 17 (2004).
- [3] NIGAM, K., MCCALLUM, A. K., THRUN, S., AND MITCHELL, T. Text classification from labeled and unlabeled documents using em. *Machine learning* 39, 2 (2000), 103–134.
- [4] ZHANG, R., FANG, R., ZHANG, W., GAO, P., LI, K., DAI, J., QIAO, Y., AND LI, H. Tip-adapter: Training-free clip-adapter for better vision-language modeling. *arXiv preprint arXiv:2111.03930* (2021).
- [5] ZOU, Y., YU, Z., KUMAR, B., AND WANG, J. Unsupervised domain adaptation for semantic segmentation via class-balanced self-training. In *Proceedings of the European conference on computer vision (ECCV)* (2018), pp. 289–305.
- [6] ZOU, Y., YU, Z., LIU, X., KUMAR, B., AND WANG, J. Confidence regularized self-training. In *Proceedings of the IEEE/CVF international conference on computer vision* (2019), pp. 5982–5991.